

[Dreamhack.io] pwn_patch_1 Write-Up

- Platform: Dreamhack
- Date: 2026-08-27 (solved) / 2026-08-28 (written)
- Difficulty: Medium
- Tags: `#pwn` `#shellcode` `#Double-Free` `#UAF` `#out_of_bound`

[!] 이 문제는 `pwn_patch_2` 문제를 풀고 나서 푸는 문제입니다.

1. Challenge Overview (문제 개요)

📄 문제 요약

- 목표: 프로그램 내 존재하는 취약점을 패치
- 제약 조건: 원본 바이너리와 크기 및 입출력 동일할 것

- 제공 파일: `origin_bin`
- 보호 기법 (Checksec):

```
Arch:      amd64-64-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x400000)
FORTIFY:   Enabled
Stripped:  No
```

2. Vulnerability Analysis (취약점 분석)

소스 코드 / 디컴파일 분석

```
undefined8 main(EVP_PKEY_CTX *param_1)
{
    int local_c [3];

    init(param_1);
    while( true ) {
        while( true ) {
            while( true ) {
                menu();
                __isoc99_scanf(&DAT_00400ab7,local_c);
                if (local_c[0] != 2) break;
                delete();
            }
            if (local_c[0] != 3) break;
```

```

        show();
    }
    if (local_c[0] != 1) break;
    add();
}
return 0;
}

```

```

void add(void)
{
    long *p1Var1;
    void *pvVar2;
    int iVar3;
    uint local_c;

    __printf_chk(1,"Size: ");
    __isoc99_scanf(&DAT_00400ab7,&local_c);
    if (local_c == 0) {
        /* WARNING: Subroutine does not return */
        exit(0);
    }
    p1Var1 = &DAT_006010a8;
    iVar3 = 1;
    if (ptr == 0) {
        iVar3 = 0;
LAB_004008b2:
        pvVar2 = malloc((ulong)local_c);
        (&ptr)[iVar3] = (long)pvVar2;
        __printf_chk(1,"Data: ");
        read(0,(void *)(&ptr)[iVar3],(ulong)(local_c - 1));
    }
    else {
        do {
            if (*p1Var1 == 0) goto LAB_004008b2;
            iVar3 = iVar3 + 1;
            p1Var1 = p1Var1 + 1;
        } while (iVar3 != 10);
    }
    return;
}

```

```

void delete(void)
{
    int local_c [3];

    __printf_chk(1,"Idx: ");
    __isoc99_scanf(&DAT_00400ab7,local_c);
    free((void *)(&ptr)[local_c[0]]);
    return;
}

```

```
void show(void)
{
    int local_c [3];

    __printf_chk(1,"Idx: ");
    __isoc99_scanf(&DAT_00400ab7,local_c);
    __printf_chk(1,"Data: %s\n",(&ptr)[local_c[0]]);
    return;
}
```

- `add()` : 사용자가 입력한 크기에 맞는 청크를 최대 10개 생성할 수 있다.
- `delete()` : 사용자가 입력한 인덱스의 청크를 `free` 한다.
- `show()` : 사용자가 입력한 인덱스의 데이터를 출력한다.

⚠ `delete()` 와 `show()`에서 사용자가 입력한 인덱스를 검사하지 않는다.
 이로 인해, 이미 해제된 청크를 다시 해제할 수 있고, (*Double Free Vuln*)
 해제된 청크의 데이터를 출력할 수 있다. (*Use-After-Free Vuln*)
 또한 입력된 값이 10 이상일 때, 다른 메모리 공간을 참조할 수 있다. (*OOB Vuln*)

💡 바이너리가 컴파일된 OS 환경은 `ubuntu 18.04 LTS` 이다. 이 환경에서는 `glibc 2.27` 버전을 사용하는데, 이 버전에서는 `tcache bin`이라는 개념이 존재한다.

하지만 `tcache safe linking` 개념이 없고, 만일 해제된 청크의 데이터를 수정할 수 있다면, `Tcache Poisoning` 공격도 가능할 것이다.

취약점 원인 (Root Cause)

- 발생 원인: 입력값 경계 검사 미흡
- 파급 효과: Double Free, Use-After-Free, OOB

3. Exploit Strategy (최종 해결 전략)

✓ 돌파구 (Breakthrough)

사용자가 입력한 값에 대해 경계 검사를 진행하고, `free` 이후에는 `NULL` 포인터로 초기화함으로써 패치를 진행한다.

- `pwn_patch_2` 문제처럼 명령어를 삽입할 수 있는 `Code Cave`를 찾아본다.

00400c80	14	00	00	00	24	01	00	00	f8	fd	ff	ff	02	00	00	00
00400c90	00	00	00	00	00	00	00	00	00	00	00	00				
00600e10	b0	07	40	00	00	00	00	00								
00600e18									90	07	40	00	00	00	00	00

- 위의 사진은 Ghidra를 통해 `.eh_frame` 영역의 끝부분을 바이너리 형태로 확인한 것이다.

- 여기서 `0x400c90 ~ 0x600e10` 까지의 영역이 사용되지 않는 영역임을 알 수 있다.

```

pwndbg> vmmap
LEGEND: STACK | HEAP | CODE | DATA | WX | RODATA

```

Start	End	Perm	Size	Offset	File (set vmmap-prefer-relpaths on)
0x400000	0x401000	r-xp	1000	0	origin_bin
0x600000	0x601000	r--p	1000	0	origin_bin
0x601000	0x602000	rw-p	1000	1000	origin_bin
0x755557c00000	0x755557c28000	r-xp	28000	0	/usr/lib/x86_64-linux-gnu/libc.so.6

- 해당 영역이 실행 가능한지 확인하기 위해 `vmmap` 명령을 통해 확인해보았다.
=> `0x400c90 ~ 0x401000` 까지의 영역이 실행 가능함을 알 수 있다.
- 이는 총 `0x370` 바이트 (880 바이트)의 명령어를 넣을 수 있음을 알 수 있다.

4. Trial & Error (삽질 및 실패 기록)

× Attempt 1: [파이썬 pwntools를 통해 수정해보자] >

- 가설: pwntools 라이브러리의 `write` 함수를 이용해 바이너리를 주입해보자
- 시도 내용:
 - `.eh_frame` 내부에 들어갈 어셈블리 명령어를 만든 뒤, `write(주소, 어셈블리 명령어)` 를 통해 바이너리에 값을 주입하자
- 결과 및 에러: `call`, `jmp` 등의 명령어의 인자를 설정하지 못함
- 원인 분석:
 - `call`, `jmp` 등 상대 주소 지정 방식의 명령어의 인자는 `offset`인데, 해당 명령어가 위치할 주소를 정확히 알지 못하는 상태에서 `offset`을 구해 넣는 건 어려웠다.

5. Exploit Code (최종 코드)

- 위의 방법이 안 되었으니, 그냥 바이너리 자체를 수정해 보는 방법으로 바꾸기로 하였다.
- 그러면 이제 바이너리의 hex 값을 어떻게 바꿀 수 있는지가 문제가 되었다.
- 처음에는 `vi` 에디터 내부에서 `:%!xxd`, `:%!xxd -r` 를 통해 문제를 풀어보았지만, 원본 데이터 랑 1바이트 차이가 났다.
- 이 1바이트는 `vi` 에디터가 추가한 `0xa` 개행 문자로 파일을 열고 저장하여 닫을 때 넣는 것이라고 한다. 이를 방지하기 위해서는 `:set noeoL` 등의 방법을 써야 한다.
- `vi` 에디터로 하면 맨 뒤에 추가되는 개행 문자를 지워야하는 번거로움이 있어서 다른 방식을 생각해보았다.
- 이때, 예전에 윈도우 PE 파일 포맷을 공부했을 때 썼던 `Hexeditor` 프로그램을 리눅스에 다운받아 패치를 진행하게 되었다.
- 어셈블리 언어를 기계어 16진수로 변환하는 것은 <https://defuse.ca/online-x86-assembler.htm> 사이트의 도움을 받았다.



delete() 함수 패치 부분

- `delete()` 함수에서 `free()` 를 호출하는 부분을 `.eh_frame` 내부로 점프하는 구문으로 변경하여 패치를 진행한다.

```

00000930 00 E8 7A FD FF FF 48 63 44 24 0C 48 8B 3C C5 A0 ..z...HcD$.H.<...
00000940 10 60 00 E9 68 03 00 00 48 83 C4 18 C3 48 83 EC ..`...h...H...H..
00000950 18 BE C1 0A 40 00 BF 01 00 00 00 B8 00 00 00 00 ....@.....
00000960 E8 2B FD FF FF 48 8D 74 24 0C BF B7 0A 40 00 B8 .+...H.t$....@..

```

커서 부분이 원래 E8 xx xx xx xx 로 delete() 에서 free() 를 호출하는 명령어 부분인데, 이것을 파일 오프셋 0xCB0 (.eh_frame 에서 끝 부분)으로 점프하는 구문으로 바꿨다.

- .eh_frame 에서는 다음의 흐름대로 명령어를 적어주었다.

```

; 사용자 입력값을 가지고 온다.
mov eax, DWORD PTR [rsp + 0xC]
; 입력값과 9를 비교한다.
cmp al, 0x9
; 만약 크다면 아래 과정을 생략하고 원래 함수로 되돌아간다.
ja 0x25
; free 함수를 호출한다.
call 0xffffffff97fe8
; 사용자 입력값을 가지고 온다.
movsxd rax, DWORD PTR [rsp + 0xC]
; 입력값에 8을 곱한다
shl rax, 3
; 동적 할당 주소를 가지고 있는 위치를 NULL로 초기화한다.
mov QWORD PTR [0x6010a0 + rax], 0x0
; 원래 함수로 되돌아간다.
jmp 0xffffffffc6ee9

```

```

00000CB0 8B 44 24 0C 3C 09 0F 87 19 00 00 00 E8 7F F9 FF .D$.<.....
00000CC0 FF 48 63 44 24 0C 48 C1 E0 03 48 C7 80 A0 10 60 .HcD$.H...H....
00000CD0 00 00 00 00 00 E9 6E FC FF FF 00 00 00 00 00 00 00 00 .....n.....
00000CE0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....

```

- 위의 어셈블리 코드를 바이너리로 바꾼 뒤, 파일 오프셋 0xCB0 위치에 적은 모습이다.

💡 show() 함수 패치 부분

- delete() 와 비슷하게 해당 함수에서 printf() 를 호출하는 부분을 .eh_frame 내부로 점프하는 구문으로 변경하여 패치를 진행한다.

```

00000980 14 C5 A0 10 60 00 BE C7 0A 40 00 BF 01 00 00 00 ....`....@.....
00000990 B8 00 00 00 00 E9 F6 02 00 00 48 83 C4 18 C3 48 .....H...H
000009A0 83 EC 18 B8 00 00 00 00 E8 29 FE FF FF B8 00 00 .....).
000009B0 00 00 E8 5E FE FF FF 48 8D 74 24 0C BF B7 0A 40 ...^...H.t$....@

```

커서 부분은 원래 printf() 를 호출하는 E8 xx xx xx xx 였지만, 그것을 E9 xx xx xx xx 로 바꿈으로써 파일 오프셋 0xC90 위치 (.eh_frame 내부)로 점프를 하게 패치했다.

- .eh_frame 내부에는 다음 흐름의 명령어가 들어간다.

```

; 사용자 입력값을 가지고 온다.
mov eax, DWORD PTR [rsp + 0xC]
; 입력값과 9를 비교한다.
cmp al, 0x9
; 만약 9보다 크다면 아래 과정을 생략하고 원래 함수로 되돌아간다.
ja 0x1b
; printf 함수를 호출한다.
call 0xffffffff9e5e8

```

```
; 원래 함수로 되돌아간다.  
jmp 0xffffffffceae9
```

```
00000C90  8B 44 24 0C 3C 09 0F 87 0F 00 00 00 90 90 90 90  .D$.<.....  
00000CA0  90 90 90 90 90 90 E8 E5 F9 FF FF E9 EA FC FF FF  .....
```

- 위의 어셈블리 코드를 바이너리로 바꾼 뒤, 파일 오프셋 0xC90 위치에 적은 모습이다.

② 명령어 도중 0x90이 있는 이유는 무엇인가

원래 `show` 함수의 경우에는 인덱스가 0과 9 사이의 값이더라도, `QWORD PTR [0x6010a0 + offset]` 위치의 값이 `Null`인 경우도 `printf` 함수 호출을 생략하고 되돌아갈 수 있도록 하였다.

그렇게 패치한 뒤, 인코딩 값을 서버에 넘겨주니, `show` 함수에 대해 검사하는 도중 `TIME OUT!` 이 뜨길래, 이걸 제거했더니 정상적으로 검사가 됨을 확인하였다.

그래서 위 사진에서 0x90 (NOP)이 많이 있는 것도 이 부분이 원래 `QWORD PTR [0x6010a0 + offset]` 위치의 값을 검사하여 분기하는 명령이었기 때문이다.

- 패치한 뒤, 다음 코드를 실행해 결과를 서버에 보낸 결과이다.

```
from pwn import *  
import base64  
  
host = 'host3.dreamhack.games'  
port = 17306  
  
p = remote(host, port)  
  
with open('patched_bin', 'rb') as f:  
    result = base64.b64encode(f.read())  
  
p.sendlineafter(b'ELF..', result)  
  
p.interactive()
```

```
[*] Switching to interactive mode  
Binary size: 9232  
[+] Correct binary size  
[+] Success to Function check.  
[+] Success to over alloc check.  
[+] Success to DFB check.  
[+] Success to delete idx check.  
[+] Success to show idx check.  
DH{ }  
[*] Got EOF while reading in interactive  
$
```

6. 배운 점 & 오답 노트

- 새로 알게 된 내용:

- vi 에디터가 추가하는 0xa : 파일을 열거나 닫을 때 마지막에 추가하는 개행 문자
- 리눅스 HexEdit 프로그램: 윈도우 환경처럼 GUI 환경은 아니지만, 바이너리를 직접 수정할 때 유용하게 사용할 것 같음